

8/6/2026

Wireshark Video

What is Wireshark :-

Wireshark is a tool used for packet capture and analysis.

It grabs all the network communications passing across your network interface, not just the ones meant for you.

Used to be called Ethereal earlier. It's open source.

For Ex :- If you want to know what is actually happening in your network, applications can sometimes lie or give unreliable logs, but the wire doesn't lie. Once data is on the wire, it's the truth.

Network Architecture :-

Before capturing packets we should understand how the network is built.

Hub :-

If laptop A sends a message to laptop B through a hub, the hub sends it to every interface it has.

So not only B gets it, but C and everyone else also gets it.

Good for sniffing, bad for performance.

Switch :-

A switch only sends traffic to the intended recipient.

If A sends to B, only B gets the message. Others won't get it.

Better for performance, but harder to sniff.

How to capture in a Switched Network :-

1. Set up a port span on the switch (needs admin access).
2. Use ARP spoofing — pretend to be other devices so the traffic comes to you.
3. Capture only your own traffic (no spoofing needed).

ARP Spoofing :-

Every device that has network capability has a MAC address (also called Physical Address).

In ARP spoofing, we pretend that we are other devices by sending out ARP responses where no ARP request was actually made.

We basically tell the network — "I am every device on this network" — and the traffic ends up coming to us.

Tools :- ettercap (GUI), arpspoof (comes with dsniff package).

Note :- Only do this where you have permission. Otherwise it is illegal.

Starting a Capture :-

Three ways to start :-

1. Click the shark fin icon in the toolbar.
2. Double-click an interface name on the welcome screen.

3. Use the menu (Capture:-Start).

The welcome screen shows the list of interfaces with a small traffic graph next to each one. The interface with traffic will show spikes in the graph.

For Ex :- Loopback interface is local-only traffic. en0 or eth0 is the actual wired interface.

Frames vs Packets :-

We commonly say "packet capture" but what Wireshark actually captures is frames.

Frame :- Layer 2 unit (the way data comes across the network interface).

Packet :- Layer 3 unit (IP layer).

Datagram :- Used for UDP.

Segment :- Used for TCP.

Different names for the same thing depending on which layer you're talking about.

Layers in a Captured Frame :-

When you click on a frame, the middle pane breaks it down layer by layer from bottom to top :-

1. Frame info — number, size, capture time.

2. Ethernet Header (Layer 2) — source MAC, destination MAC, type field. This type field tells us what comes next.

3. IP Header (Layer 3) — source IP, destination IP, TTL, fragmentation flags, protocol field (tells us what next).

4. TCP / UDP Header (Layer 4) — source port, destination port.

5. Application Data (Layer 7) — the actual HTTP request, DNS query, etc.

Each layer has a field that tells the receiver what protocol comes next. That's how the receiving end knows how to parse the next chunk.

Capture Filters vs Display Filters :-

Capture Filter :- Decides what gets recorded in the first place.

Display Filter :- Decides what you SEE from what is already recorded.

Capture Filter Examples :-

host 172.30.42.13:-only traffic to/from this IP

port 80:-only port 80 traffic

tcp:-only TCP traffic

Display Filter Examples :-

ip.addr == 172.30.42.13:-IP can be in source OR destination

ip.src == 172.30.42.13:-only when source matches

http:-only HTTP traffic

dns:-only DNS traffic

http.response.code == 404:-find 404 errors

http.request.method == "GET":-only GET requests

Filter box turns green when valid, pink when invalid. Helpful indicator.

You can chain filters using and, or, not.

Following Streams :-

Right-click a packet:-Follow:-TCP Stream.

This rebuilds the entire conversation between two endpoints, strips away the headers and shows you only the data going back and forth.

Red text :- What you sent.

Blue text :- What came back.

Wireshark assigns each stream a number (tcp.stream eq 520) so you can filter on it later.

Works for UDP too, even though UDP is technically stateless.

(9/6/20206)

Statistics :-

Wireshark can pull out a lot of statistics from a capture.

Endpoints :-

List of every IP, MAC, and port that showed up. One-sided view — basically "who appeared".

Conversations :-

Pairs of endpoints that communicated. Shows packet count, byte count, duration.

Sort by bytes to find the noisiest pairs in the network.

Protocol Hierarchy :-

Statistics :- Protocol Hierarchy gives a tree view of every protocol that appeared with percentage of packets and bytes.

Good for spotting protocols you didn't expect.

For Ex :- If you see IRC messages on a corporate network, that's suspicious.

Expert Information :-

Wireshark automatically detects suspicious or broken stuff in the capture.

Bottom-left corner has a colored circle showing the highest severity issue :-

1.Red:-Error (something is broken)

2.Yellow:-Warning (suspicious)

3.Cyan:-Note (notable but normal)

4.Blue:-Chat (just informational)

Click the dot to see the panel.

It groups things like out-of-order packets, retransmissions, duplicate ACKs, malformed packets, port unreachable, etc.

Note :- Retransmissions aren't always bad. TCP retransmits normally if it doesn't get ACKs in time. But lots of retransmits = network problem.

Color Codes in the Packet List :-

Green:-HTTP

Light Blue:-DNS

Black background with Red text:-Error (check sum problem, duplicate ACK, etc.) — pay attention!

Black background:-Other issue (like ICMP port unreachable)

Can be customized in View:-Coloring Rules.

Name Resolution :-

By default Wireshark shows raw IP addresses and port numbers. These can be resolved to human-readable names.

1.Network Address:-IP becomes hostname (reverse DNS).

2.Transport Name:-port number becomes service name (80:-http, 22:-ssh).

3.MAC Address:-vendor name (Apple, Samsung, etc.).

Best practice :- Keep resolution OFF during capture (it generates extra DNS traffic that clutters the capture). Turn it ON after capture, while viewing.

GeoIP — Mapping IPs to Locations :-

To know where in the world an IP address is located.

1.Download MaxMind GeoLite databases (country, city, ASN).

2.Unzip the .dat files.

3.Tell Wireshark the folder path in Preferences:-Name Resolution:-GeoIP.

4.Enable lookups in Preferences:-Protocols:-IPv4.

Now you can use filters like `ip.geoip.country == "United States"`.

Statistics:-Endpoints will show country, city, latitude, longitude for each IP.

There's also a Map button that generates an actual map with dots.

Useful for spotting traffic going to unexpected countries (data exfiltration etc.).

Saving and Exporting :-

1.File:-Save:-save the whole capture as .pcap (older format, wider compatibility) or .pcapng (newer Wireshark format).

2.Export only displayed packets:-useful after narrowing down with a display filter.

3.Export selected packets:-only the ones you've highlighted.

4.File:-Export Objects:-HTTP / SMB:-pulls out actual files that were transferred over the network.

For Ex :- If someone downloaded a JPEG during the capture, Wireshark can rebuild the JPEG from packets and save it to your disk. Very useful in CTF challenges with .pcap files.

Investigating Latency :-

Wireshark doesn't store absolute time per packet — only relative time from start of capture.

To measure timing between messages :-

1.Preferences:-Protocols:-TCP:-enable "Calculate conversation timestamps".

2.Now each TCP packet will have "Time since first frame in this TCP stream" and "Time since previous frame in this TCP stream".

3.Right-click that field:-Apply as Column to see it in the main list.

4.Sort by it to find the slowest exchanges.

If you see times consistently small and then suddenly spike, that means congestion or a slow server.

Command Line — tshark :-

tshark is the command line version of Wireshark.

Useful when you are SSH'd into a server and don't have a GUI.

Basic Usage :-

tshark:-start capturing on default interface

tshark -i en0:-pick interface

tshark -w saved.pcap:-write to file

tshark -s 0:-capture full packet (no truncation)

tshark -f "host 172.30.42.13":-capture filter

tshark -c 500:-stop after 500 packets

tshark -a duration:15:-stop after 15 seconds

Extracting Specific Fields :-

tshark -T fields -e frame.number -e ip.src -e ip.dst > output.txt

Spits out only the fields you ask for, tab-separated.

Can be imported into Excel for further analysis.

Statistics from Command Line :-

tshark -q -z io,phs:-protocol hierarchy

tshark -q -z hosts:-list of all hosts seen

tshark -q -z http,tree:-HTTP statistics

Use tshark -z help if you forget the syntax.

editcap and mergecap :-

editcap :- Splits a big pcap into smaller chunks.

editcap -c 5000 input.pcap split.pcap:-creates files of 5000 packets each.

Can also split by time, remove duplicates, chop packet size, etc.

mergcap :- Combines multiple pcaps into one.

mergcap -w combined.pcap split*.pcap:-interleaves them by timestamp.

Useful when ring buffer captures created many small files and you want one big file.

VoIP and Telephony :-

Wireshark can analyze voice calls over the network.

Two main protocols :-

SIP (Session Initiation Protocol) :-

Text-based, very HTTP-like. Easy to read raw.

Messages :- INVITE, 100 trying, 180 ringing, 200 OK, ACK, BYE, etc.

H.323 :-

Binary protocol. Has several sub-protocols (H.225, H.225 RAS, etc.).

Not human-readable raw — you NEED Wireshark to dissect.

RTP (Real-time Transport Protocol) :-

Both SIP and H.323 use RTP to carry the actual audio.

VoIP Tools in Wireshark :-

1.Telephony:-VoIP Calls:-lists all detected calls with timestamps.

2.Telephony:-SIP Flows:-Flow Sequence:-generates ladder diagrams of message flow.

3.RTP Streams:-Analyse:-gives jitter, latency, lost packet stats (= call quality).

4.RTP Streams:-Play Streams:-plays back the audio if not encrypted.

Ladder diagrams are very useful for VoIP troubleshooting. Easier than tracing messages mentally.

Final Note :-

Wireshark is basically a microscope for networks. You install it, capture traffic, and suddenly you can see things that were invisible — what apps actually send, where they send it, whether it's broken.

The real skill is not Wireshark itself, it's networking. Once you understand TCP handshakes, DNS, HTTP, TLS handshakes, Wireshark just confirms what you already know.

Next Step :- Download Wireshark, capture your own traffic, follow some TCP streams from real websites, play with display filters.

Also grab sample pcaps from wiki.wireshark.org/SampleCaptures or pcapr.net to practice with protocols you don't see on your own network (SIP, H.323, SCTP, etc).